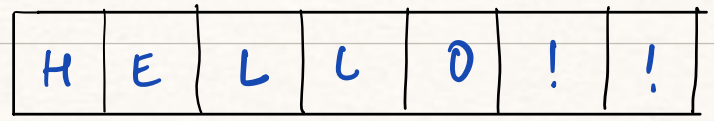
COL 1101

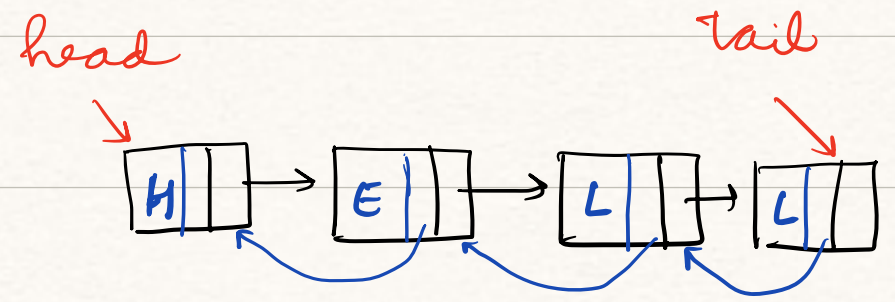
LECTURE 9

AUGUST 11, 2026

STACKS AND QUEUE

ARRAY AND LINKEDLIST

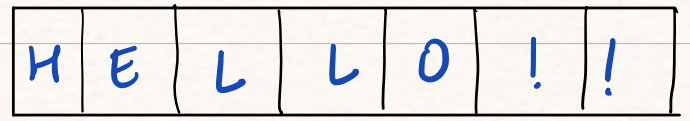


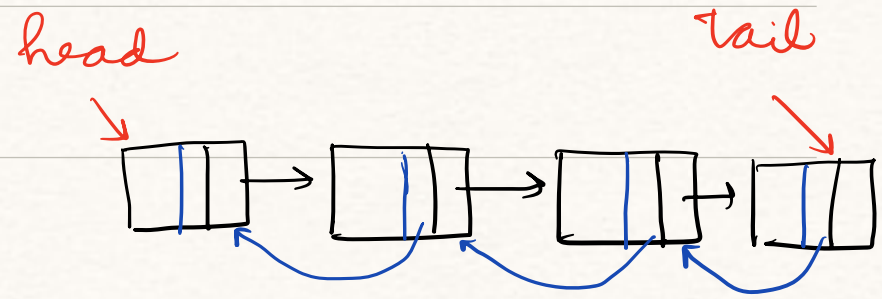


Array: contiguous memory

Linked list : Each node contains reference to next (prev) node.

ARRAY AND LINKEDLIST





Array: contiguous memory

Random access is cheap

Insert / delete are expensive

Linked list : Each node contains reference to next (prev) node.

Insert / delete is cheap

Random access is expensive

Which of them is better ?

SCENARIO #1

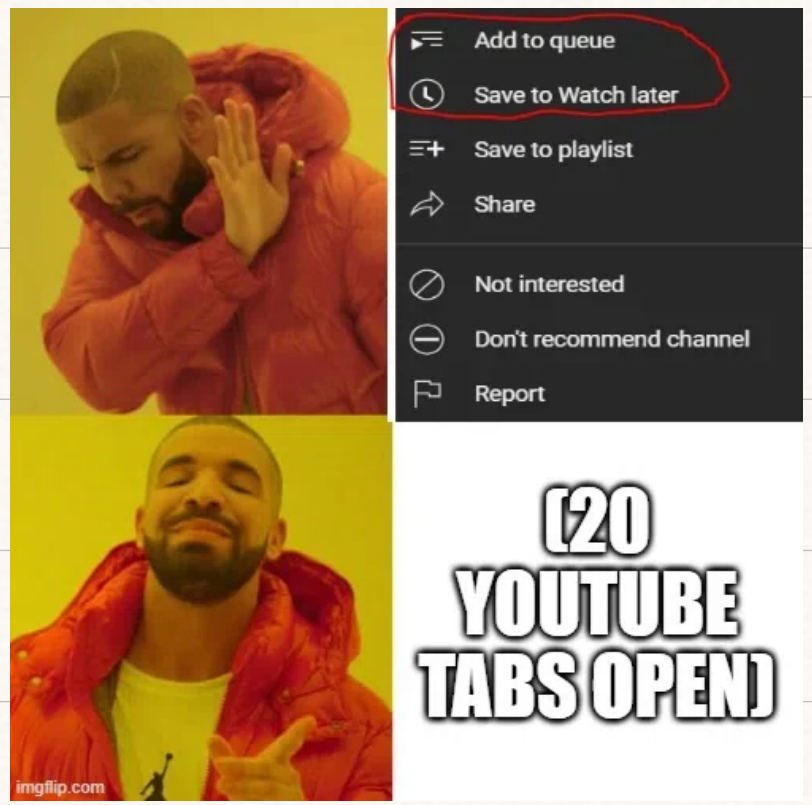
Leaderboard: Your server hosts a battle royale game where exactly 100 players compete, and you need to track and display their live rankings on demand. Players frequently want to click on the 45th or 82nd ranked player to instantly load and view their specific profile.



SCENARIO #2

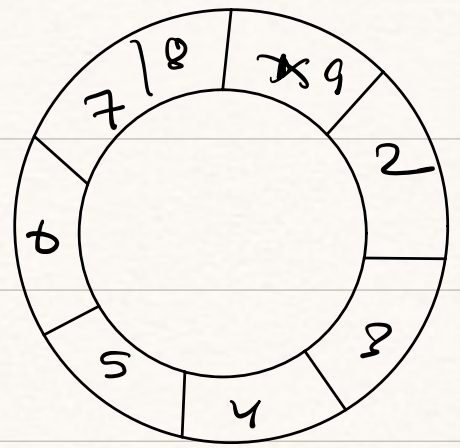
Youtube Playlist: You hold the CURRENT track. The user hits previous / next to move either way, and can remove the current track or insert one right after it.

linked list



SCENARIO #3

A fixed the oldest, 8-slot ring of recent search terms: bounded size, you overwrite and you index into it.



Array

WHAT WE DID

We ask what is better in this case.

WHAT WE DID

We ask what is better in this case.

what is the data access pattern determines

the choice of the data structure.

PRACTICAL TIP #1

What is the time complexity of a linear scan ;

i.e. in-order traversal over all elements of -

1). Array → $\Theta(n)$

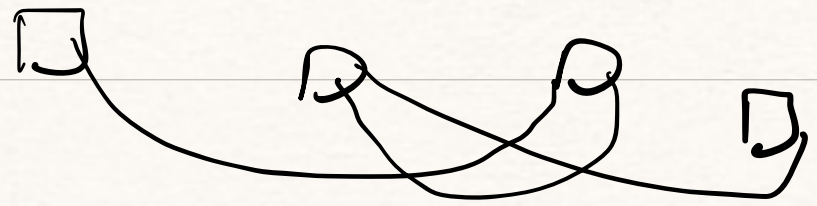
2). Linked list → $\Theta(n)$

PRACTICAL TIP #1

What is the time complexity of a linear scan ?

i.e. in-order traversal over all elements of -

1). Array



2). Linked list

Wall clock time for n = 1,000,000 elements (linear scan)

Traversal	Best time (ms)	vs list
list (contiguous)	21.3	1.0x
linked (tight-built)	31.9	1.5x
linked (scattered)	123.4	5.8x

PRACTICAL TIP #1: CACHE LOCALITY ALSO MATTERS

- Array references sit side-by-side
- Linked list nodes live scattered

While a linear scan through both of them is $\Theta(n)$, the wall-clock time can differ

PRACTICAL TIP #1: CACHE LOCALITY ALSO MATTERS

- Array references sit side-by-side
- Linked list nodes live scattered

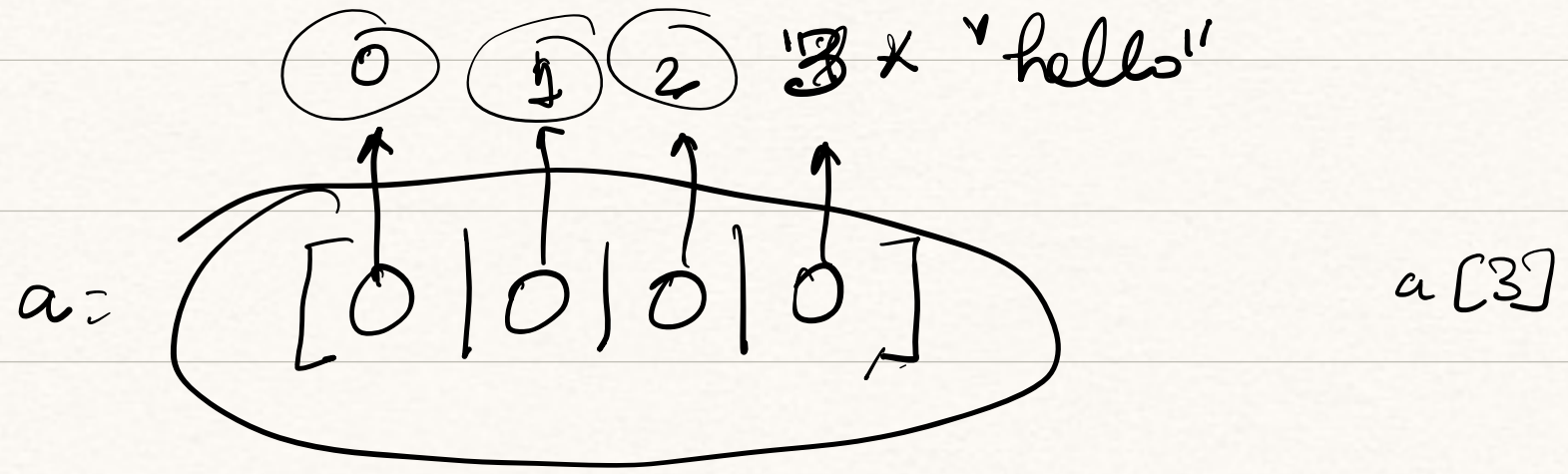
While a linear scan through both of them is $\Theta(n)$, the wall-clock time can differ

PRACTICAL TIP # 2 : SPACE GAP IN PYTHON

- Python's list (array) is a compact block of references

```
import sys
sys.getsizeof([])                    # 56   -- the empty container
sys.getsizeof([0])                   # 64   -- 56 + one 8-byte slot
sys.getsizeof([0, 1, 2, 3])          # 88   -- 56 + 4 slots
```

sys.getsizeof([0, 1, 2, "hello"])



PRACTICAL TIP #2 : SPACE GAP IN PYTHON

- Python's list (array) is a compact block of references

```
import sys
sys.getsizeof([])              # 56   -- the empty container
sys.getsizeof([0])             # 64   -- 56 + one 8-byte slot
sys.getsizeof([0, 1, 2, 3])    # 88   -- 56 + 4 slots
```

What is getsizeof measuring here?

SPACE USED BY LINKED LIST

```
class Node:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next

n = Node(0)
sys.getsizeof(n)                # ~48   -- the instance itself...
sys.getsizeof(n.__dict__)       # ~296  -- ...plus a per-instance dict!
```

> 300 bytes

OPTIMIZING SPACE : USE __slots__

```python
class Node:
    __slots__ = ("value", "next")      # no per-instance __dict__
    def __init__(self, value, next=None):
        self.value = value
        self.next = next
```

8 bytes ← self.prev =

```python
sys.getsizeof(Node(0))          # ~48   -- header + value ref + next ref
```

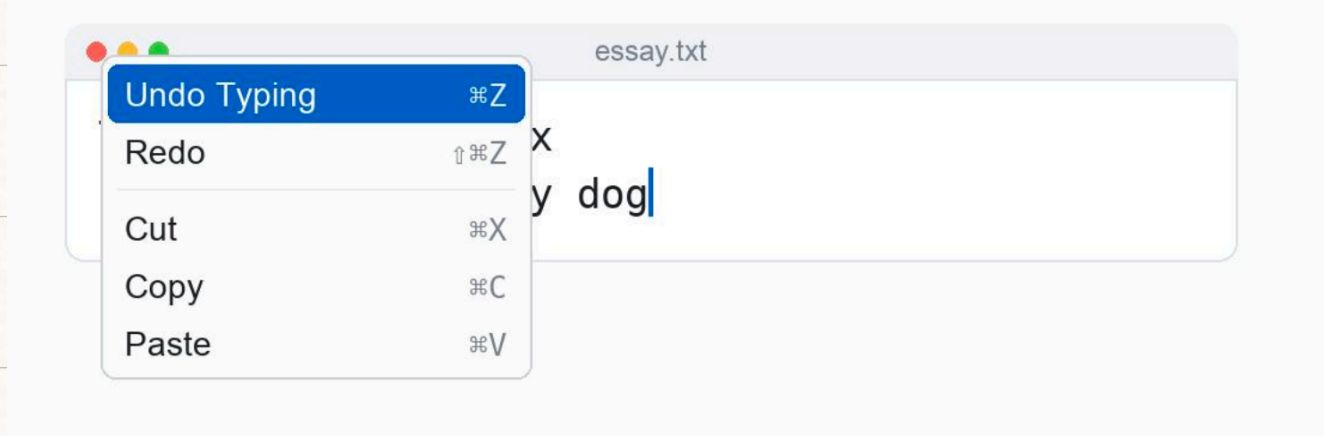
n = Node(0) n.n1 = 2 + 8 = 56

→ __slots__ deletes the dict

→ how much for double linked list ?

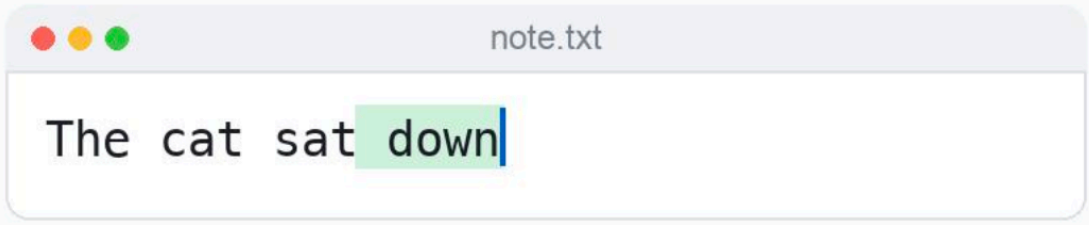
STACKS AND QUEUES

UNDO FUNCTIONALITY



How does the editor implement Undo?

IMPLEMENTING UNDO FUNCTIONALITY



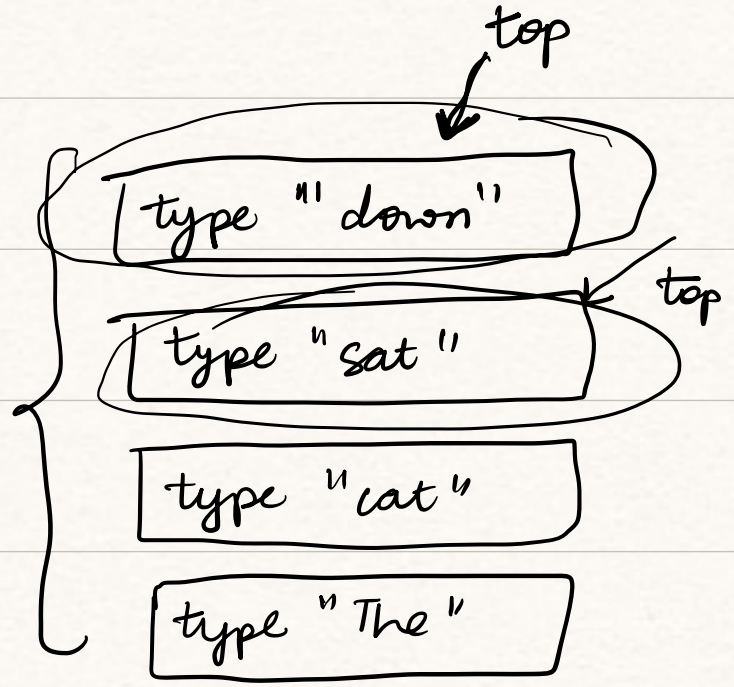
The note can be thought of as a sequence of edits.

edit =

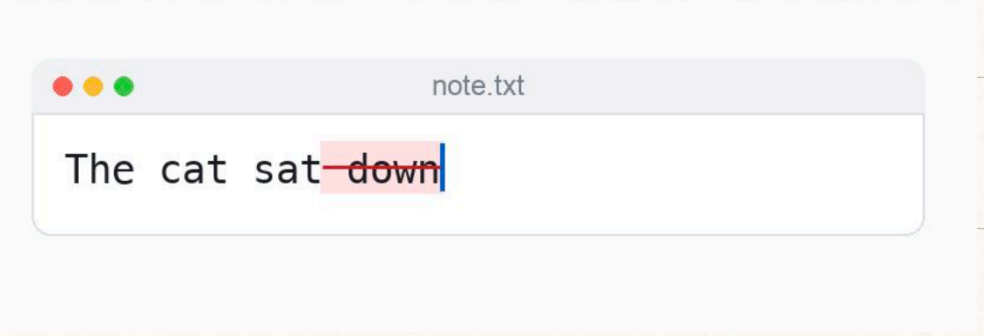
undo: ↑ remove the most recent edit β

→ reverse(edit)

sequence of edits



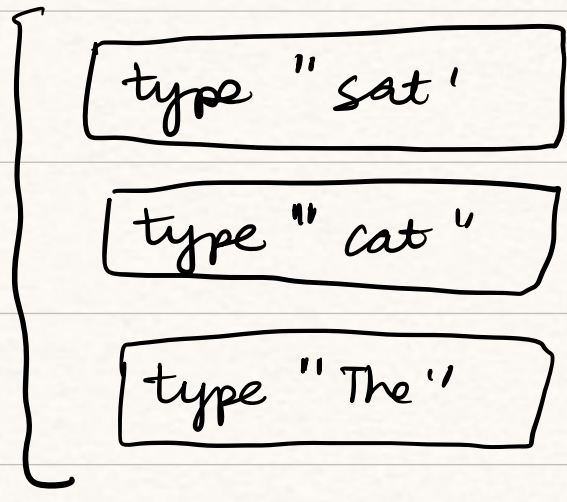
IMPLEMENTING UNDO FUNCTIONALITY



Undo takes the most recent edit and reverses it

→ Last-in, first-out (LIFO)

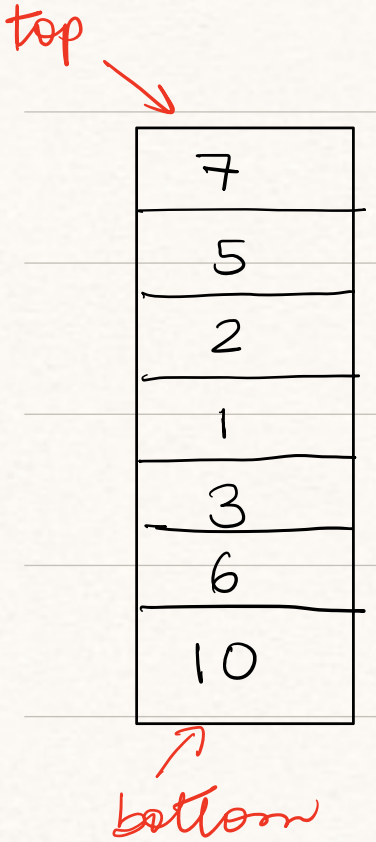
Stack



THE STACK ADT

- Stores data as a list of elements

- Constraints: data can be read / deleted / inserted at one end of the list

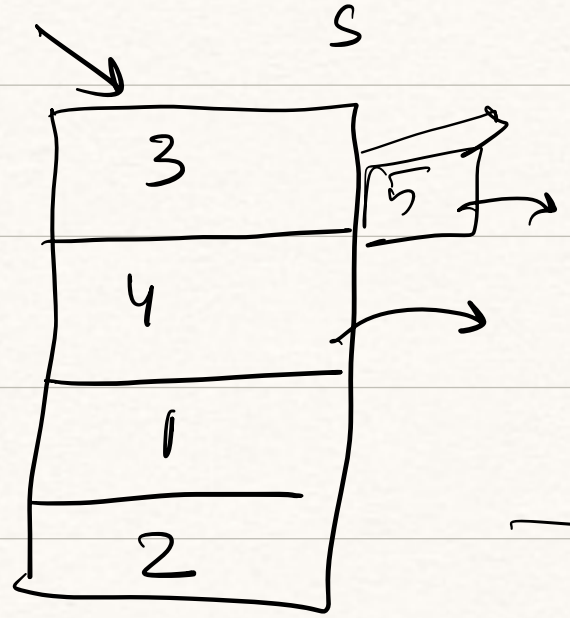


Operations

- push (x): put x on the top
- pop(): remove and returns the top
- peek(): return the top, unchanged
- size(): returns the number of elements

How to implement it ?

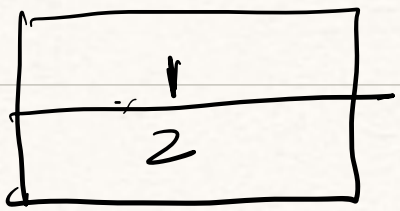
top



s.pop()

s.push(5)

s.pop() x2



How do you implement it?

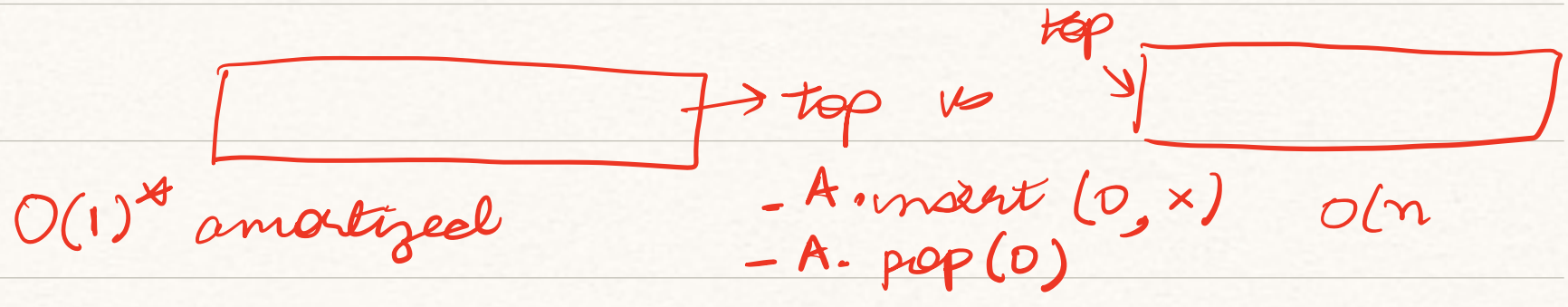
IMPLEMENTING STACKS USING ARRAYS (PYTHON LIST)

The top is the end of the list

```
class ArrayStack:
    def __init__(self):
        self._A = []

    def push(self, x):    self._A.append(x)
    def pop(self):        return self._A.pop()
    def peek(self):       return self._A[-1]
    def size(self):   return len(self._A)
```

self._A



IMPLEMENTING STACKS USING ARRAYS (PYTHON LIST)

The top is the end of the list

```
class ArrayStack:
    def __init__(self):
        self._A = []

    def push(self, x):    self._A.append(x)
    def pop(self):        return self._A.pop()
    def peek(self):       return self._A[-1]
    def size(self):   return len(self._A)
```

Why top NOT the beginning of the list ?

LinkedList._Node (2)

IMPLEMENTING STACK USING LINKED LIST

```
class LinkedStack:
    class _Node:
        __slots__ = ("value", "next")
        def __init__(self, value, next=None):
            self.value = value
            self.next = next

    def __init__(self):
        self._head = None
        self._size = 0

    def size(self):
        return self._size
```

class LLI

class Node

head

How to implemet push / pop / peek ?

```
def push(self, x):
    self._head = self._Node(x, self._head)
    self._size += 1

def pop(self):
    n = self._head
    self._head = n.next
    self._size -= 1
    return n.value

def peek(self):
    return self._head.value
```

Implementing Stack using linked list

```
class LinkedStack:
    class _Node:
        __slots__ = ("value", "next")
        def __init__(self, value, next=None):
            self.value = value
            self.next = next

    def __init__(self):
        self._head = None
        self._size = 0

    def size(self):
        return self._size
```

```
def push(self, x):          # prepend
    self._head = self._Node(x, self._head)
    self._size += 1

def pop(self):              # drop the head
    node = self._head
    self._head = node.next
    self._size -= 1
    return node.value

def peek(self):
    return self._head.value
```

TIME COMPLEXITY

Operation	ArrayStack	LinkedStack
push	$\Theta(1)^*$	$\Theta(1)$
pop	$\Theta(1)^*$	$\Theta(1)$
peek	$\Theta(1)$	$\Theta(1)$
size	$\Theta(1)$	$\Theta(1)$

* amortized